

Universidad Galileo	Guatemala 5 de octubre 2026
Facultad: FISICC	Alumno: Luis Cordova
Curso: Microprocesadores	Carnet: 23012122
Sección: A	Hora de Laboratorio: 1:00 a 1:50
Auxiliar: Brandon Eduardo Mendoza Espinoza	Día de Laboratorio: Lunes

Laboratorio #7 Puertos Paralelos en Lenguaje Ensamblador

Resumen:

Esta práctica fue repetir el laboratorio #4, el parpadeo del LED de usuario con la interrupción del SysTick, pero esta vez las dos subrutinas del GPIO iban en ensamblador Thumb2 [4]. El main.c quedó sin tocar: solo declara LEDs_Init y LED_Toggle como externas, arma el SysTick y llama a LED_Toggle en cada interrupción.

La tarea era completar GPIO.s. Primero hubo que ubicar en el manual de referencia las direcciones base de los puertos y los offsets de MODER y ODR, y después escribir las dos subrutinas siguiendo el esquema de leer, modificar y escribir del código de ejemplo. Como punto extra se puso en uno el bit 18 de RCC->AHB2ENR por bit banding, con una sola escritura y sin leer el registro. Ese punto no se llegó a presentar en el laboratorio por falta de tiempo, ya que la práctica se publicó tarde; se hizo después y su funcionamiento aparece en el video.

Objetivos:

- Completar las subrutinas LEDs_Init y LED_Toggle en ensamblador Thumb2 para que la tarjeta repita el resultado del laboratorio #4.
- Entender cómo se enlazan C y ensamblador: el extern del lado de C, el EXPORT del lado del ensamblador y el uso de los registros al llamar una subrutina.
- Usar bit banding para modificar un solo bit de un registro de periférico con una sola instrucción de escritura.

Teoría:

El Cortex-M4 del STM32G431 implementa ARMv7-M, que solo ejecuta Thumb2, con instrucciones de 16 y 32 bits mezcladas [7]. Es una arquitectura de carga y almacenamiento: las operaciones aritméticas y lógicas solo trabajan sobre registros, así que para cambiar un registro de periférico hay que traerlo con ldr, modificarlo en un registro del CPU y devolverlo con str [3].

Un proyecto puede mezclar C y ensamblador porque ambos terminan como archivos objeto que el enlazador une. Para que funcione, el ensamblador publica el nombre de la subrutina con EXPORT y el archivo en C la declara con extern, que le avisa al compilador que la función existe pero está en otro archivo [5]. Además, los dos lados deben respetar la convención de llamada de ARM (AAPCS): los argumentos van en r0 a r3, el resultado vuelve en r0, la subrutina puede usar r0 a r3 y r12 sin guardarlos, y r4 a r11 deben quedar como estaban [6].

En ensamblador no hay estructuras de CMSIS como GPIOB->MODER, así que las direcciones se definen a mano con la directiva EQU, que le pone nombre a un número [5]. La dirección de un registro se arma sumando la base del periférico más el offset del registro, y la instrucción ldr r1, [r0,#MODER] hace justo esa suma [1].

Bit banding es una función del Cortex-M4 que asigna a cada bit de la región de periféricos, de 0x40000000 a 0x400FFFFF, una palabra completa en una región alias que arranca en 0x42000000. Esa región alias no es memoria real: cuando el CPU escribe en una de sus direcciones, el bus la traduce a la dirección del registro y al bit correspondiente, y es el hardware el que hace la lectura, la modificación y la escritura de ese bit, sin que el programa pueda ser interrumpido en medio. Escribir un 1 en la palabra alias pone el bit en 1, escribir un 0 lo pone en 0, y leerla devuelve el valor actual del bit [3]. La dirección alias se calcula así [3]:

$$\text{alias} = 0x42000000 + (\text{dirección} - 0x40000000) \times 32 + \text{bit} \times 4$$

Resultados prácticos:

A continuación se presentan las capturas obtenidas durante el desarrollo y la ejecución de la práctica.

```

Lab7_LuisCordova_23012122
main.c GPIO.s X
G4_Test1 > ASM GPIO.s
1      AREA |.text|, CODE, READONLY, ALIGN=2
2      THUMB
3      REQUIRE8
4      PRESERVE8
5
6
7      EXPORT  LEDs_Init
8      EXPORT  LED_Toggle
9
10     RCC      EQU      0x40021000
11
12     ; Deben buscar el valor en el Reference Manual
13     GPIOE     EQU      0x0 ; EvalBoard
14     ;GPIOB     EQU      0x48000400 ; K8
15     GPIOB     EQU      0x48000400 ; G4
16     GPIOA     EQU      0x0 ; RE
17
18     AHB2ENR    EQU      0x0000004C
19
20     ; Deben buscar el valor en el Reference Manual
21     MODER      EQU      0x00
22     ODR        EQU      0x14
23
24     ; Puntos extra: Bit Banding
25     BIT_N      EQU      18
26     BB_AHB2ENR EQU      0x42000000 + ((RCC + AHB2ENR - 0x40000000) * 32) + (BIT_N * 4)
27
28

```

Ilustración 1- símbolos definidos en GPIO.s, incluidos los del bit banding.

```
LEDs_Init
push {lr}

;RCC->AHBENR |= (1<<1);
;*** Encender el puerto correspondiente de tu tarjeta ***

ldr r0, =RCC          ;read
ldr r1, [r0,#AHB2ENR]
movs r2, #0x2          ;modify
orrs r1, r1, r2
str r1, [r0,#AHB2ENR] ;write

;EvalBoard / Discovery
; GPIOE->MODER |= (1<<26);
; GPIOE->ODR &= ~(1<<13);
;F303K8
; GPIOB->MODER &= ~(1<<7);
; GPIOB->MODER |= (1<<6);
;G431KB
; GPIOB->MODER &= ~(1<<17);
; GPIOB->MODER |= (1<<16);
;F303RE
; GPIOA->MODER |= (1<<10);
; GPIOA->ODR &= ~(1<<5);

ldr r0, =GPIOB          ;read
ldr r1, [r0,#MODER]
ldr r2, =0x00020000      ;modify (1<<17)
bics r1, r1, r2
ldr r2, =0x00010000      ;(1<<16)
orrs r1, r1, r2
str r1, [r0,#MODER]      ;write
```

Ilustración 2- inicio de LEDs_Init: reloj del puerto B y configuración de PB8 como salida.

```

ip  <  >  Lab7_LuisCordova_23012122

C main.c  ASM GPIO.s  X

G4_Test1 > ASM GPIO.s
29  LEDs_Init
58      ldr  r2, =0x00010000      ;(1<<16)
59      orrs r1, r1, r2
60      str  r1, [r0,#MODER]      ;write
61
62      ;RCC->AHB2ENR bit n = 1 con Bit Banding (puntos extra)
63      ldr  r0, =BB_AHB2ENR
64      movs r1, #1
65      str  r1, [r0]
66
67      pop  {pc}
68
69  LED_Toggle
70      push {lr}
71
72      ;GPIOE->ODR ^= 1<<13 EvalBoard / Discovery
73      ;GPIOB->ODR ^= 1<<3 K8
74      ;GPIOB->ODR ^= 1<<8 G4
75      ;GPIOA->ODR ^= 1<<5 RE
76
77      ldr  r0, =GPIOB            ;read
78      ldr  r1, [r0,#ODR]
79      ldr  r2, =0x00000100      ;modify (1<<8)
80      eors r1, r1, r2
81      str  r1, [r0,#ODR]        ;write
82
83      pop  {pc}
84
85      end
86

```

Ilustración 3- final de LEDs_Init con la escritura por bit banding, y subrutina LED_Toggle.

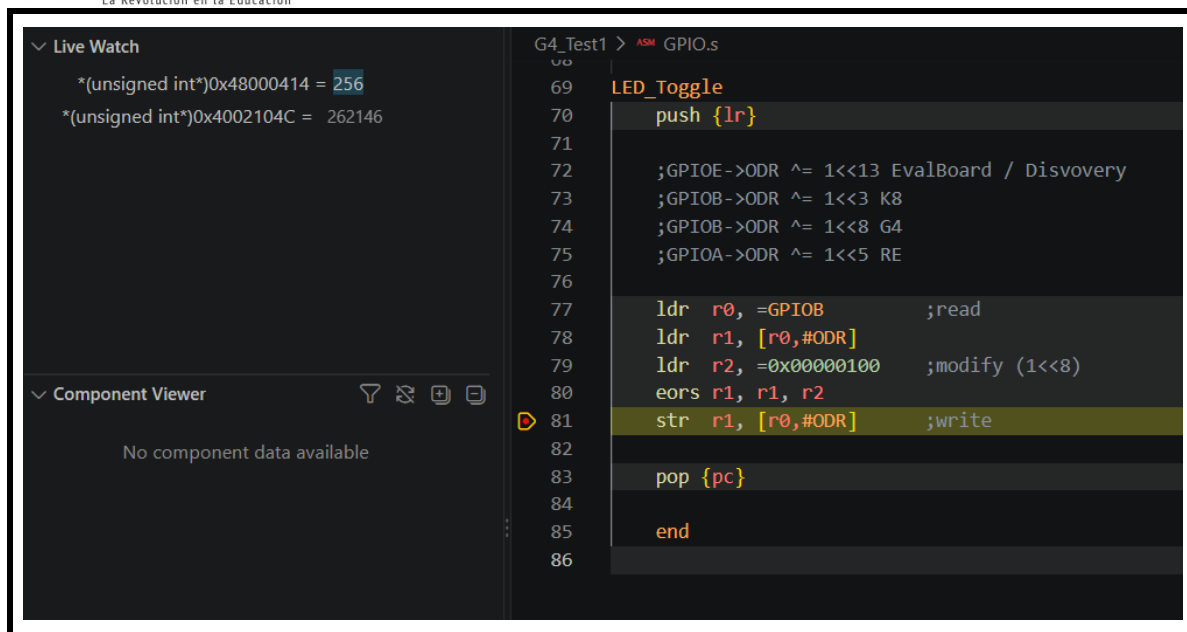


Ilustración 4- depurador detenido en LED_Toggle: GPIOB->ODR vale 256 (bit 8 en uno) y RCC->AHB2ENR vale 262146 (bits 1 y 18).

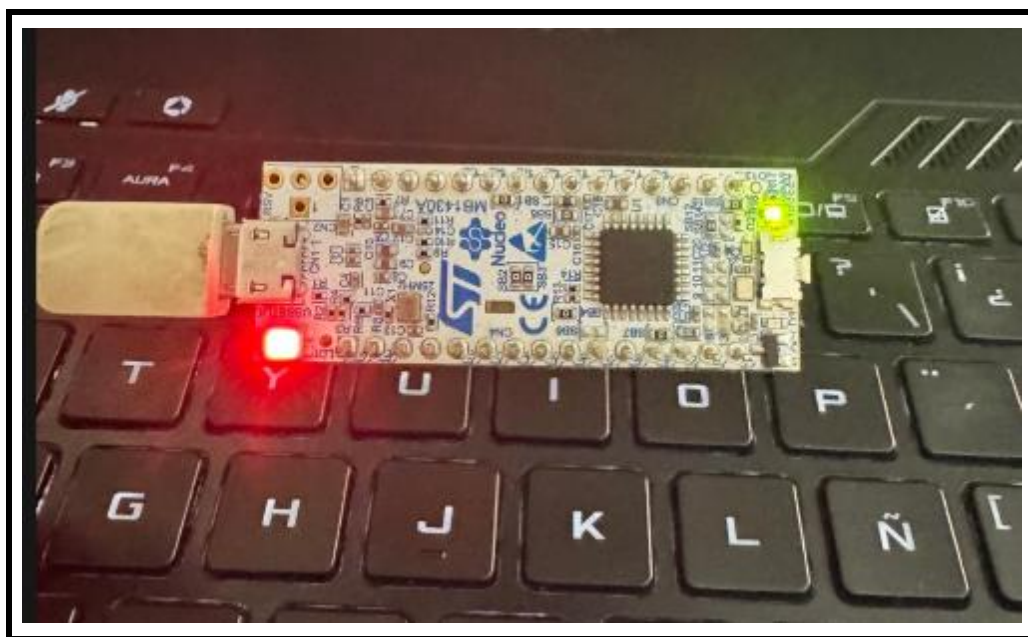


Ilustración 5- tarjeta Nucleo-G431KB ejecutando el programa.

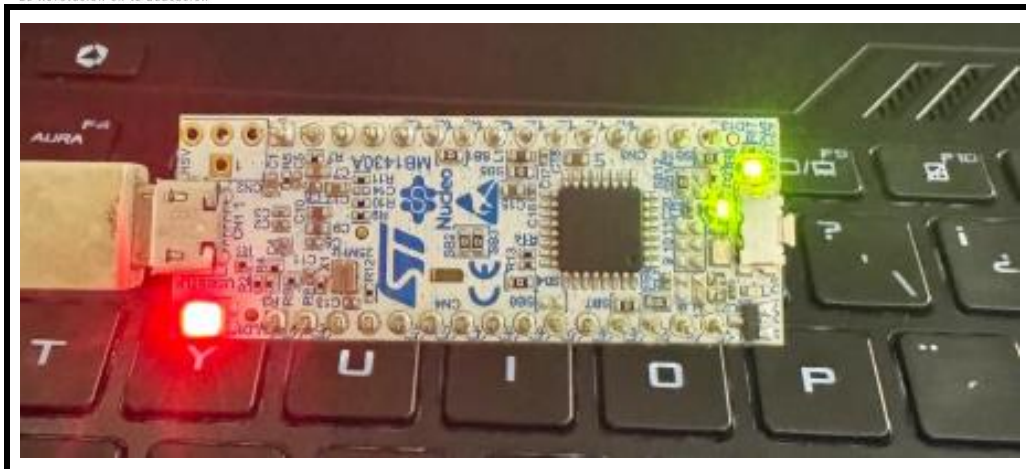


Ilustración 6- tarjeta Nucleo-G431KB con el LED LD2 parpadeando.

Video de la práctica:

En el siguiente enlace se muestra el funcionamiento de la aplicación en la tarjeta.

<https://youtu.be/ekoZCGZFMUk>

Resultados Teóricos:

Preguntas del paso 5

¿Por qué las funciones declaradas en main.c utilizan el modificador extern?

Porque LEDs_Init y LED_Toggle no están escritas en main.c. El extern le avisa al compilador que esas funciones existen en otro archivo, así que compila las llamadas sin quejarse y deja la dirección pendiente para que el enlazador la complete. En C las funciones ya son externas por defecto, así que el modificador no cambia el código generado, pero deja claro que la definición está en otro lado [5].

¿Dónde están definidas esas subrutinas?

En GPIO.s. Cada una es una etiqueta seguida de sus instrucciones, y las dos se publican con EXPORT LEDs_Init y EXPORT LED_Toggle; sin esas directivas el enlazador no las encontraría y daría un error de símbolo no definido.

¿Ya se dio cuenta de que estamos replicando el laboratorio #4?

Sí. En el laboratorio #4 el programa en C configuraba el SysTick y en su rutina de interrupción llamaba a LED_Toggle para invertir el LED LD2 en PB8 [2]. Aquí el main.c hace exactamente lo mismo; lo único que cambia es que LEDs_Init y LED_Toggle ahora están en ensamblador.

Símbolos (paso 6)

Las direcciones base y los offsets se tomaron del mapa de memoria y de las tablas de registros del manual de referencia del STM32G4 [1]:

Símbolo	Valor	Qué es
RCC	0x40021000	base del módulo de reloj (ya venía)
AHB2ENR	0x4C	offset del registro de reloj del GPIO (ya venía)
GPIOB	0x48000400	base del puerto B, el del LED LD2
GPIOA	0x0	puerto de la F303RE, no se usa
GPIOE	0x0	puerto de la EvalBoard, no se usa
MODER	0x00	offset del registro de modo
ODR	0x14	offset del registro de salida

Cada línea del archivo corresponde a una tarjeta distinta, y la Nucleo-G431KB solo usa la de G4. El archivo traía dos líneas GPIOB, una para la F303K8 y otra para la G431KB; un símbolo de EQU no se puede definir dos veces, y como en las dos tarjetas el puerto B está en la misma dirección, la línea de la K8 se dejó como comentario. GPIOA y GPIOE pertenecen a la F303RE y a la EvalBoard, el programa no los usa y se dejaron en 0x0 como venían; en el mapa de memoria del STM32G4 sus direcciones serían 0x48000000 y 0x48001000 [1].

Subrutinas (paso 7)

LEDs_Init ya traía el encendido del reloj del puerto B: lee RCC->AHB2ENR, le hace OR con 0x2, que es el bit 1 (GPIOBEN), y lo vuelve a escribir. Después se agregó la configuración de PB8 como salida, traduciendo las dos líneas de C que da el comentario para la G431KB:

C	Ensamblador
GPIOB->MODER &= ~(1<<17);	ldr r0,=GPIOB / ldr r1,[r0,#MODER] / ldr r2,=0x00020000 / bics r1,r1,r2
GPIOB->MODER = (1<<16);	ldr r2,=0x00010000 / orrs r1,r1,r2 / str r1,[r0,#MODER]
GPIOB->ODR ^= 1<<8;	ldr r1,[r0,#ODR] / ldr r2,=0x00000100 / eors r1,r1,r2 / str r1,[r0,#ODR]

Las dos operaciones sobre MODER se hacen con una sola lectura y una sola escritura: bics apaga el bit 17 y orrs enciende el 16, así que los bits 17 y 16 quedan en 01, que es salida. Limpiar el bit 17 sí hace falta en esta tarjeta, porque después del reset GPIOB->MODER vale 0xFFFFFEBF y PB8 arranca en 11, modo analógico [1]. LED_Toggle hace la tercera línea: lee ODR, invierte el bit 8 con eors y lo escribe.

Las dos subrutinas solo usan r0, r1 y r2, que según la AAPCS se pueden modificar sin guardarlos [6]. El push {lr} al inicio y el pop {pc} al final guardan la dirección de retorno y regresan con ella al punto desde donde se llamó la subrutina.

Antes de cargar el programa se ensambló la misma secuencia para el Cortex-M4 y se revisó el código máquina: los valores 0x20000, 0x10000 y 0x100 entran como constantes inmediatas de una instrucción mov.w, y las direcciones de RCC y de GPIOB quedan guardadas en una tabla de literales al final del código, de donde las toma ldr r0,=.

Funcionamiento (paso 8)

main.c llama a SysTick_Config(SystemCoreClock/50). Con el reloj interno de 16 MHz, el SysTick cuenta 320 000 ciclos y genera una interrupción cada 20 ms. En cada una, LED_Toggle invierte el LED, así que pasa 20 ms encendido y 20 ms apagado: un periodo de 40 ms, o 25 Hz, que se ve como un parpadeo muy rápido, igual que en el laboratorio #4.

Para comprobarlo se puso un punto de ruptura en la instrucción str de LED_Toggle y se agregaron al Live Watch las expresiones `*(unsigned int*)0x48000414`, que es `GPIOB->ODR (0x48000400 + 0x14)`, y `*(unsigned int*)0x4002104C`, que es `RCC->AHB2ENR (0x40021000 + 0x4C)`. Con cada ejecución del str, ODR pasa de 256 a 0 y de 0 a 256, es decir, el bit 8 cambia de estado, y el LED se apaga y se prende al mismo tiempo (ilustración 4).

Bit banding (paso 10)

Este punto no se llegó a presentar en el laboratorio por falta de tiempo, porque la práctica se publicó tarde. Se implementó después con $n = 18$ y su funcionamiento se muestra en el video.

El paso pide modificar el bit n de `RCC->AHBENR`. En el STM32G4 los puertos GPIO están en el bus AHB2, así que el registro equivalente es `RCC->AHB2ENR`, en `0x4002104C`. Con la fórmula de la teoría, su alias empieza en `0x42420980` y cada bit avanza 4 bytes. En el STM32G431 no todos los bits de ese rango existen: el microcontrolador solo tiene DAC1 y DAC3, así que los bits 17, 19 y 20 son reservados y siempre se leen como cero [1]:

n	Dirección alias	Bit en AHB2ENR (G431)
16	0x424209C0	DAC1EN
17	0x424209C4	reservado
18	0x424209C8	DAC3EN
19	0x424209CC	reservado
20	0x424209D0	reservado

Por eso se escogió $n = 18$. El cálculo de su dirección alias es:

$$\begin{aligned} & 0x42000000 + (0x4002104C - 0x40000000) \times 32 + 18 \times 4 \\ & = 0x42000000 + 0x2104C \times 32 + 72 \\ & = 0x42000000 + 0x420980 + 0x48 \\ & = 0x424209C8 \end{aligned}$$

En GPIO.s el número se pone en el símbolo BIT_N, y el símbolo BB_AHB2ENR calcula la dirección con la misma fórmula al ensamblar. LEDs_Init solo agrega tres instrucciones:

```
ldr r0, =BB_AHB2ENR
movs r1, #1
str r1, [r0]
```

La primera instrucción carga la dirección alias 0x424209C8, la segunda pone un 1 en r1 y la tercera lo escribe. Esa escritura enciende únicamente el bit 18, sin leer el registro y sin tocar el bit 1 del GPIOB, que ya estaba encendido. Con el método normal harían falta cinco instrucciones, como las que encienden el reloj del puerto B: cargar la dirección, leer, crear la máscara, hacer el OR y escribir; y si una interrupción cambiara el registro entre la lectura y la escritura, ese cambio se perdería.

El resultado se comprobó en el depurador con la expresión `*(unsigned int*)0x4002104C` en el Live Watch. Antes de la escritura, RCC->AHB2ENR vale 2, que es solo el bit 1, el reloj del puerto B. Después vale 262146, que es la suma del bit 1 (2) y el bit 18 (262144), o 0x00040002 en hexadecimal (ilustración 4). El bit 18 habilita el reloj del DAC3, que el programa no usa, así que encenderlo no afecta el parpadeo.

Conclusiones:

- Escribir LEDs_Init y LED_Toggle en ensamblador mostró qué hace en realidad cada línea de C sobre un registro: una línea como `GPIOB->ODR ^= 1<<8` se convierte en cargar la dirección, leer el registro, invertir el bit y escribirlo de vuelta.
- C y ensamblador conviven en el mismo proyecto gracias a tres acuerdos: `extern` en C, `EXPORT` en el ensamblador y la convención de llamada AAPCS, que permite usar r0 a r3 sin guardarlos y regresar con la dirección guardada en lr.
- Bit banding cambia un solo bit con una sola escritura, sin leer el registro, lo que evita perder cambios si una interrupción ocurre en medio; a cambio, la dirección alias hay que calcularla y solo sirve para un bit a la vez.

Bibliografía:

- [1] STMicroelectronics, «STM32G4 series advanced Arm-based 32-bit MCUs», Reference manual RM0440, Rev. 9, marzo de 2025. [En línea]. Disponible en: https://www.st.com/resource/en/reference_manual/rm0440-stm32g4-series-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
- [2] STMicroelectronics, «STM32G4 Nucleo-32 board (MB1430)», User manual UM2397, Rev. 2, septiembre de 2019. [En línea]. Disponible en: https://www.st.com/resource/en/user_manual/um2397-stm32g4-nucleo32-board-mb1430-stmicroelectronics.pdf
- [3] STMicroelectronics, «STM32 Cortex-M4 MCUs and MPUs programming manual», Programming manual PM0214. [En línea]. Disponible en: https://www.st.com/resource/en/programming_manual/pm0214-stm32-cortexm4-mcus-and-mpus-programming-manual-stmicroelectronics.pdf
- [4] J. Fajardo, «Práctica de Laboratorio #7: Puertos Paralelos en Lenguaje Ensamblador», Microprocesadores, Universidad Galileo, octubre de 2026.
- [5] Arm Limited, «armasm User Guide», Arm Compiler 6. [En línea]. Disponible en: <https://developer.arm.com/documentation/>
- [6] Arm Limited, «Procedure Call Standard for the Arm Architecture (AAPCS32)». [En línea]. Disponible en: <https://github.com/ARM-software/abi-aa/blob/main/aapcs32/aapcs32.rst>
- [7] Arm Limited, «ARMv7-M Architecture Reference Manual». [En línea]. Disponible en: <https://developer.arm.com/documentation/ddi0403/latest/>